

การบริหารความเสี่ยง

Sopify AI-Assisted SDLC — วิธีการประเมิน ทะเบียนความเสี่ยง การควบคุม และการยกระดับปัญหา

Document ID	RISK-SDLC-001	Version	1.1.0 — Final
วันที่	2026-06-11	ผู้รับผิดชอบ	ฐากรัฐ วิทยาศาสตร์ไพบูลย์ (RD)
Asana task	Docs : Risking — [RD-M-26-O7-Q1-KR6.4/AI] Sopify & Risk Management AI Strategy	ขอบเขตการใช้	ทุกโปรเจกต์ภายใต้ skill sopify-sdlc

1. วัตถุประสงค์และขอบเขต

เอกสารนี้กำหนดวิธีการระบุ ประเมิน ควบคุม และยกระดับ (escalate) ความเสี่ยงตลอดวงจรการพัฒนาซอฟต์แวร์แบบใช้ AI ช่วย (Sopify AI-assisted SDLC) ซึ่งผู้ใช้ที่ไม่ใช่นักพัฒนา (non-developer) สร้างและส่งมอบเว็บแอปพลิเคชันภายในของ GS Battery ผ่าน Claude Cowork, GitHub, Supabase และ Railway

ครอบคลุมทุกเฟสของ SOP-SDLC — Requirements, Design, Database, Development, Testing, Deploy และ Git Workflow ที่พาดผ่านทุกเฟส — รวมถึงรูปแบบการทำงานแบบ multi-agent (PM, UXUI Design, Frontend Engineer, Backend Engineer, Tester) และบังคับใช้กับทุกโปรเจกต์ที่ดำเนินการภายใต้ skill sopify-sdlc (SOP-SDLC, Code Standard SOP-DEV-001)

นอกขอบเขต: ความเสี่ยงด้าน IT ระดับองค์กร (เครือข่าย, อุปกรณ์หลายทาง, ความปลอดภัยทางกายภาพ) ซึ่งอยู่ในความรับผิดชอบของฝ่าย IT ของ GS Battery ตามนโยบายแยกต่างหาก

2. วิธีการประเมิน — 5x5 โอกาสเกิด x ผลกระทบ

ความเสี่ยงแต่ละรายการให้คะแนนเป็น Risk Score = โอกาสเกิด (L) x ผลกระทบ (I) โดยแต่ละด้านให้คะแนน 1-5

โอกาสเกิด (Likelihood)

ระดับ	ชื่อเรียก	ความหมาย
1	แทบไม่เกิด (Rare)	แทบเป็นไปไม่ได้ในการทำโปรเจกต์ปกติ
2	ไม่น่าเกิด (Unlikely)	อาจเกิดได้ในสถานการณ์พิเศษเท่านั้น
3	เป็นไปได้ (Possible)	อาจเกิดขึ้นในบางโปรเจกต์
4	น่าจะเกิด (Likely)	คาดว่าจะเกิดในโปรเจกต์ส่วนใหญ่หากไม่มีการควบคุม
5	เกิดแน่นอน (Almost certain)	จะเกิดขึ้นแน่นอนหากไม่มีการควบคุม

ผลกระทบ (Impact)

ระดับ	ชื่อเรียก	ความหมาย
1	เล็กน้อยมาก (Negligible)	ผลกระทบผิวเผิน ไม่มีงานแก้ไข ไม่มีผลต่อข้อมูลหรือความปลอดภัย
2	เล็กน้อย (Minor)	งานแก้ไขเล็กน้อยภายในเฟสเดียว
3	ปานกลาง (Moderate)	งานแก้ไขข้ามเฟส หรือกระทบกำหนดการ
4	รุนแรง (Major)	เกิด defect ใน production, ระบบล่ม หรืองานแก้ไขจำนวนมาก
5	รุนแรงมาก (Severe)	ข้อมูลรั่วไหล ข้อมูลสูญหาย หรือเหตุการณ์ด้านความปลอดภัย

ระดับความเสี่ยง (Risk level bands)

คะแนน	ระดับ	การตอบสนองที่ต้องทำ
15-25	สูง (High)	ต้องมี mitigation และตรวจสอบยืนยันที่ gate เสมอ; ยกระดับทันทีเมื่อเกิดเหตุ
8-12	กลาง (Medium)	ต้องมี mitigation; ติดตามที่ phase gate
1-6	ต่ำ (Low)	ยอมรับได้ภายใต้การควบคุมมาตรฐาน; ทบทวนในรอบ

ทะเบียนความเสี่ยงต้องถูกทบทวนทุกครั้งที่ตัดสินใจ phase gate และปรับฐานใหม่ (re-baseline) เมื่อเริ่มโปรเจกต์และก่อนปล่อยแต่ละ release (Release Gate 2)

3. ทะเบียนความเสี่ยง (Risk Register)

ความเสี่ยงที่ระบุได้ 14 รายการ — L = โอกาสเกิด, I = ผลกระทบ, S = คะแนน

ความเสี่ยงระดับสูง (คะแนน 15-25)

ID	ความเสี่ยง	L	I	S	ผู้รับผิดชอบ	Mitigation
R-01	การรั่วไหลของ secret — Supabase service key, JWT secret หรือ .env ถูก commit ลง repo หรือถูกวางใน context/แชต	4	5	20	IT + User (orchestrator)	เก็บ secret ใน env vars / Railway variables เท่านั้น; ห้าม commit .env (ใช้ .env.example เท่านั้น); IT ส่ง token ผ่านช่องทางที่ปลอดภัย; reviewer ตรวจสอบ secret ทุก PR; หากรั่วไหลให้ทำ key rotation ตามหัวข้อ 5 ทันที
R-02	การข้าม phase gate — เริ่มงาน downstream uu artifact ต้นทางที่ยังไม่ผ่านการอนุมัติ (การทำงานแบบ non-linear ไม่ได้ยกเว้น gate)	4	4	16	User (orchestrator)	Gate คือจุดตรวจความครบถ้วนเดียวก่อน Push; Tester ตรวจสอบยืนยันทุก gate กับไฟล์จริงอย่างอิสระ; อ่าน ledger docs/PHASE_STATE.md จากดิสก์ก่อนการมอบหมายหรือตัดสินใจ gate ทุกครั้ง; ผลผ่านที่ worker รายงานเองไม่เพียงพอ
R-03	การแก้ schema โดยไม่ได้รับอนุมัติ — เขียนหรือรัน SQL ก่อนที่เอกสาร schema จะผ่านการอนุมัติ	3	5	15	Backend Engineer agent	เอกสาร schema มาก่อน SQL เสมอ — Backend agent ต้องหยุดรอการอนุมัติจาก user ผ่าน orchestrator; บันทึกการอนุมัติลง ledger; ทำ migration ผ่าน Supabase MCP เท่านั้น พร้อมทดสอบ UP/DOWN
R-04	AI สร้าง requirement เกินจริง (hallucination) — agent แต่ง story, AC หรือขอบเขตที่ user ไม่เคยขอ แล้วเฟสถัดไปสร้างงานต่อจากสิ่งนั้น	4	4	16	PM agent + User	MoSCoW + user stories + AC ต้องมาจาก brief ของ user เท่านั้น; user อนุมัติ docs/requirements.md ก่อนเริ่ม design/database; AC ถูกใช้เป็น UAT scenario ทำให้ขอบเขตที่ถูกสร้างขึ้นถูกจับได้ที่ Gate 2
R-05	โค้ดที่ AI สร้างถูก merge โดยไม่มีการทดสอบ — โค้ดผ่านการรีวิวแบบผิวเผินแต่ไม่มีเทสหรือมี defect ซ่อนอยู่	4	4	16	Tester agent + Senior reviewer	เขียนเทสคู่กับโค้ด (coverage FE >= 70%, BE >= 80%); Gate 1 ทุก PR (lint, type-check, tests, ไม่มี debug code); Gate 2 ก่อน release (integration ครบ 100% ของ endpoint, k6 P95 < 2s, OWASP scan 0 critical); senior review พร้อม label MUST/SHOULD/NIT

ความเสี่ยงระดับกลาง (คะแนน 8-12)

ID	ความเสี่ยง	L	I	S	ผู้รับผิดชอบ	Mitigation
R-06	Context drift ("lost in the middle") — กฎ SOP ถูกละทิ้งเจือปน ๆ ใน session ที่ยาว	3	3	9	User (orchestrator)	รูปแบบ multi-agent: หนึ่งเฟส = sub-agent ใหม่หนึ่งตัวที่ได้รับ SOP slice ของเฟสนั้นแบบคำต่อคำ; ส่ง artifact เป็น path ของไฟล์ไม่วางเนื้อหายาว; ledger บนดิสก์คือแหล่งความจริงเดียว

ID	ความเสี่ยง	L	I	S	ผู้รับผิดชอบ	Mitigation
R-08	Migration ที่ไม่มี rollback ที่ทดสอบแล้ว — แก้ schema โดยไม่มี DOWN path ที่ใช้งานได้	3	4	12	Backend Engineer agent	ทุก migration ต้องมี UP และ DOWN (comment ไว้); ทดสอบทั้งคู่กับ DB เปรียบเทียบขึ้น production; หนึ่ง migration ต่อหนึ่งการเปลี่ยนแปลงเชิงตรรกะ; ดูแล schema changelog
R-09	ไม่มี RLS บนตาราง multi-tenant — ข้อมูลมองเห็นข้าม tenant/user	2	5	10	Backend Engineer agent + IT	Phase 3 gate ตรวจสอบ RLS policy; ใช้ parameterized query เท่านั้น; DB user ของแอปได้สิทธิ์น้อยที่สุด (least privilege); go-live gate ตรวจสอบ RLS ชั่วก่อน deploy
R-07	Push ตรงเข้า protected branch — งานข้าม PR, review และ CI	2	4	8	IT (repo admin)	Branch protection บน main และ develop (ต้องมี PR + CI ผ่าน + approval, ห้าม force-push); งานทั้งหมดอยู่บน branch type/asana-id-description; Railway deploy จาก main เท่านั้น งานที่ไม่ผ่านรีวิวจึงไปไม่ถึง production
R-10	ช่องโหว่ใน dependency — CVE ที่รู้จักใน npm package หลุดไปถึง production	3	3	9	Senior reviewer + IT	ตรวจสอบ dependencies checklist ตอนเริ่มโปรเจกต์; commit lockfile; OWASP scan ที่ Gate 2 ต้อง 0 critical; มี audit step ใน CI template
R-11	ตามรอยงานไม่ได้ (traceability loss) — เชื่อมโยง Asana - branch - PR - deploy ไม่ได้	3	3	9	PM agent + User	ชื่อ branch มี Asana task ID; ทุก PR ใส่ลิงก์ Asana task ใน description (เป็นข้อ DoD); ใช้ Conventional Commits; แดวใน ledger บันทึก path ของ artifact และวันที่

ความเสี่ยงระดับต่ำ (คะแนน 1-6)

ID	ความเสี่ยง	L	I	S	ผู้รับผิดชอบ	Mitigation
R-12	ไม่เป็นไปตาม brand / accessibility — UI เพี้ยนจาก brand guideline หรือใช้คีย์บอร์ดไม่ได้	2	2	4	UX/UI Design agent	Phase 2 gate: token ตรงตาม guideline, layout 375px ไม่พัง, Tab/Enter ใช้งานได้, มี async state ครบ; prompt ของ Figma Make ต้องระบุ brand guideline เสมอ
R-13	Deploy ล้มเหลว / ระบบล่มบน Railway — build หรือ config ที่ผิดพลาดไปถึง production	2	3	6	IT + User	Pre-deploy checklist (build ผ่านบนเครื่อง, tsc --noEmit, มี health check endpoint); smoke test + ตรวจสอบหลัง deploy; ถ้าไม่ผ่านให้ Railway rollback กลับ deploy ก่อนหน้า
R-14	ความรู้กระจุกตัว — ผู้ปฏิบัติงาน non-dev คนเดียวถือ context ของโปรเจกต์ทั้งหมด	2	3	6	PM agent + RD lead	Artifact และสถานะทั้งหมดอยู่ใน repo (docs/, design/, migrations, docs/PHASE_STATE.md) ไม่ใช่ในประวัติแชต; session ใดก็เริ่มงานต่อจาก repo ได้; SOP skill มี version และครบถ้วนในตัวเอง

4. การฝังการควบคุมไว้ใน Quality Gate

การควบคุมความเสี่ยงไม่ใช่ขั้นตอนตรวจสอบที่แยกออกมา — แต่ฝังอยู่ใน quality gate เดิมของ SOP การข้าม gate จึงเท่ากับถอดการควบคุมออก (ดู R-02)

จุดควบคุมตาม SOP	ควบคุมความเสี่ยง
Phase 1 gate — MoSCoW, AC ทุก story, NFRs, user อนุมัติ requirements	R-04, R-11
Phase 2 gate — brand token, responsive, accessibility, async states	R-12
Phase 3 gate — schema doc อนุมัติก่อน SQL; ทดสอบ UP/DOWN; RLS; constraints	R-03, R-08, R-09

จุดควบคุมตาม SOP	ควบคุมความเสี่ยง
Phase 4 gate — Zod ครบทุกจุด, ไม่มี 'any', ไม่มี secret, response shape มาตรฐาน	R-01, R-05
Phase 6 Gate 1 (ทุก PR) — lint, type-check, unit tests, coverage, ไม่มี debug code	R-05
Phase 6 Gate 2 (ก่อน release) — integration 100%, k6 P95 < 2s, OWASP 0 critical, UAT sign-off	R-04, R-05, R-10, R-12
Phase 7 — branch protection, PR template + ลิงก์ Asana, senior review, Conventional Commits	R-01, R-02, R-07, R-11
Phase 5 go-live gate — secret ใน Railway env, HTTPS, backups + RLS, smoke tests	R-01, R-09, R-13
Ledger docs/PHASE_STATE.md + การตรวจยืนยัน gate โดย Tester	R-02, R-06, R-14

5. เส้นทางการยกระดับปัญหา (Escalation Path)

- ต่ำ (Low) — บันทึกลงทะเบียนความเสี่ยงในการทบทวน gate ครั้งถัดไป; ใช้การควบคุมมาตรฐาน
- กลาง (Medium) — เจ้าของเฟสทำงานขึ้นที่ได้รับผลกระทบ ดำเนินการ mitigation แล้วให้ Tester ตรวจยืนยัน gate ใหม่ก่อนกลับมาทำงานต่อ; แจ้ง PM ในการอัปเดตสถานะรอบถัดไป
- สูง (High) — หยุดเฟสที่ได้รับผลกระทบทันที แจ้ง PM และ IT ภายในวันเดียวกัน; ห้ามกลับมาทำงานต่อจนกว่า Tester จะตรวจยืนยัน gate ที่เกี่ยวข้องใหม่ และบันทึกการแก้ไขลง docs/PHASE_STATE.md และ Asana task แล้ว
- Secret รั่วไหล (R-01) — ดำเนินการทันทีไม่ว่าระดับใด: หยุด push และ deploy ทั้งหมด; IT หมุนเวียนกุญแจที่รั่วทันที — regenerate Supabase anon/service keys, regenerate production JWT secret (openssl rand -base64 32 — ห้ามใช้ค่า dev ซ้ำ), อัปเดต Railway environment variables; ล้าง secret ออกจาก git history (rewrite หรือ revoke — การลบไฟล์ไม่ได้แปลว่า secret หายไป); ตรวจสอบแอปด้วยกุญแจชุดใหม่ รัน smoke test แล้วจึงกลับมาทำงานต่อ; บันทึกเหตุการณ์เป็น Asana task ที่เชื่อมโยงโปรเจกต์ พร้อมสาเหตุและการแก้ไข

6. การทบทวนและบำรุงรักษา

ทะเบียนความเสี่ยงนี้เป็นเอกสารที่มีชีวิต: ทบทวนทุกครั้งที่ตัดสินใจ phase gate, หลังเกิดเหตุการณ์ใด ๆ และก่อนปล่อยทุก release การแก้ไขที่มียุทธศาสตร์ต้องส่งผ่าน git workflow มาตรฐาน — branch docs/, Conventional Commit, PR ที่เชื่อมโยง Asana เข้าสู่ develop — เพื่อให้ฐานความเสี่ยงมี version คู่กับโค้ดที่มันปกป้อง